Lecture 2 — Advanced Data Structures: Persistent Segment Trees, Merge-Sort Trees, Treaps

> National Championship Prep · Prerequisites: segment trees (week 23), lazy propagation (week 25), coordinate compression (regional-06), ordered sets (week 13) · Estimated time: 2 × 90 min

1. Overview

A plain segment tree answers "aggregate over a range of *positions*" for the *current* array. Championship problems routinely break one of those three assumptions:

Problem shape	Example	Tool
Query a range of positions **and** a range of values	"k-th smallest in a[l..r]", "how many a[i] ≤ x in [l, r]"	persistent segment tree, merge-sort tree
Query an **old version** of the array	"array k as it was after update t", "copy array k"	persistent segment tree
The array **changes shape**	cut and paste, reverse a range, insert in the middle	implicit treap

All three are 30–60 lines of code. All three belong in your notebook (Lecture 1).

2. Persistent segment trees

2.1 Path copying

A data structure is *persistent* if every update creates a new version while keeping all old versions queryable.

A point update in a segment tree changes only the nodes on the root-to-leaf path: $\lceil \log_2 n \rceil + 1$ nodes. **Path copying**: instead of modifying those nodes, create copies of them; each copy points to the new child on the path and to the *old* child off the path. The new root represents the new version; the old root still represents the old version, untouched.

```
version 0:        r0                version 1 (update leaf 3):    r1
                /    \                                          /    \
```

```
              A        B                                    A      B'      (A is
shared)
             / \      / \                                        /  \
            1   2    3   4                                      3'   4    (4 is
shared)
```

Cost. Each update allocates exactly one node per level: O(log n) time and O(log n) memory. After q updates the structure has O(n + q log n) nodes. Queries are unchanged: start from the root of the version you want.

Because children are shared, nodes must be stored with **explicit child indices** (`L[v]`, `R[v]`), not the implicit `2v, 2v+1` layout.

Memory sizing. Pre-allocate `2n + q·(⌈log₂ n⌉ + 1)` nodes, e.g. for n = q = $2 \cdot 10^5$ about $4.2 \cdot 10^6$ nodes. Using parallel arrays of `int` children and `long long` sums keeps that around 70 MB.

2.2 Application 1: versions and copies

CSES 1737 — Range Queries and Copies: maintain a list of arrays; operations are "set a[k][i] = x", "sum of a[k][l..r]", "append a copy of array k". With path copying, a *copy is O(1)*: the new array is just the same root index. See §6.2.

2.3 Application 2: k-th smallest in a range

SPOJ MKTHNUM — K-th Number: given a static array, answer queries "k-th smallest value among a[l..r]".

Compress values to 0..m−1. Build a segment tree **over values**, where a leaf counts how many times that value occurs. Insert a[1], a[2], …, a[n] one by one, keeping a root after each insertion: `root[i]` describes the multiset {a[1..i]}.

Key fact. For any value range [x, y], `cnt(root[r], [x,y]) − cnt(root[l−1], [x,y])` is the number of indices i in [l, r] with a[i] ∈ [x, y]. Counts are *subtractable* between versions, so the pair of trees (root[r], root[l−1]) behaves like one segment tree for the subarray a[l..r].

Now walk down both trees simultaneously, as in "walking the segment tree" (week 23): at a node with left-child count `c = cnt[L[u]] − cnt[L[v]]`, if k ≤ c go left, otherwise set k −= c and go right. One root-to-leaf walk: **O(log n) per query**, O(n log n) preprocessing.

Correctness. Invariant: the answer is the k-th smallest value of a[l..r] restricted to the current value range. At the root this is the definition. If at least k elements fall in the left half, the k-th smallest is in the left half and keeps rank k; otherwise it is in the right half with rank k − c. At a leaf the range is a single value, which is the answer.

The same "difference of two prefix versions" idea answers:

- count of a[i] ≤ x in [l, r] (online version of SPOJ KQUERY);
- number of **distinct** values in [l, r] online (SPOJ DQUERY): process positions left to right; at position i, add +1 at position i and −1 at the previous occurrence of a[i]. Then version r, queried on positions [l, r], counts each value once (only its last occurrence ≤ r is +1).

2.4 Application 3: binary search over versions

Codeforces 484E — Sign on Fence: for a query (l, r, w), find the largest h such that some w consecutive planks in [l, r] all have height ≥ h. Sort heights in decreasing order and insert positions one at a time into a persistent segment tree storing "longest run of ones" (prefix/suffix/best merge). Version t contains exactly the planks with the t largest heights. The answer is the smallest t whose version has a run of length ≥ w inside [l, r] — a binary search over versions, $O(\log^2 n)$ per query.

The pattern "sort by one parameter, insert in that order, one version per prefix" turns many 2D problems into persistent segment tree problems.

3. Merge-sort trees

A **merge-sort tree** is a segment tree over positions where each node stores the **sorted list** of values in its segment — exactly the intermediate arrays of merge sort.

- **Memory/build:** each level holds n values in total, so O(n log n) memory and build time (merge the two children with `std::merge`).
- **Query** "how many a[i] ≤ x for i ∈ [l, r]": decompose [l, r] into O(log n) canonical nodes, `upper_bound` in each: **$O(\log^2 n)$**.
- **k-th smallest** with a merge-sort tree needs an extra binary search over the answer: $O(\log^3 n)$. Use the persistent tree when you need k-th.

Why still learn it? It is the simplest correct structure for "range × value" counting: no persistence bookkeeping, easy to adapt (store prefix sums next to each sorted list to get "sum of values ≤ x in [l, r]"). It also generalizes to **point updates** if each node stores an order-statistics tree (`__gnu_pbds::tree`, week 13) or a Fenwick tree over compressed values instead of a static vector: $O(\log^2 n)$ per operation.

Fractional cascading reduces the query to O(log n) by storing, for each element of a node's list, its position in both children's lists; one binary search at the root then suffices. It is rarely needed in contests, but worth knowing as the reason "merge-sort tree with pointers" appears in some editorials. A **wavelet tree** is a related O(n log σ)-memory structure that answers k-th smallest and range counting in O(log σ).

Offline alternative. If all queries are known in advance, sort the elements and the queries by value and sweep with a Fenwick tree over positions: O((n + q) log n) with tiny memory. This is the intended solution of SPOJ KQUERY. Always ask: *are the queries online?* If not, offline sweeps usually beat persistence (see also the World Finals track's offline techniques lecture).

4. Treaps and implicit treaps

4.1 Treap = BST + heap

A **treap** stores keys in BST order and assigns each node a random priority; the tree is a max-heap by priority. For a given set of (key, priority) pairs the treap is **unique**: the root is the max-priority node, its left subtree holds the smaller keys, and so on recursively.

Expected depth is O(log n). Sort the keys as $x_1 < \ldots < x_n$. Node x_i is an ancestor of x_j exactly when x_i has the highest priority among x_{min(i,j)}, ..., x_{max(i,j)}. With random priorities this happens with probability $1/(|i - j| + 1)$. The expected depth of x_j is therefore $\Sigma_{i \neq j}\ 1/(|i-j|+1) < 2 \cdot H_n \approx 2 \ln n$. All operations are expected O(log n), regardless of the input order. Adversarial inputs cannot hurt you as long as priorities come from a good random generator seeded at runtime (e.g. `mt19937` seeded with the clock).

4.2 Split and merge

Everything is built from two operations:

- `split(t, k) → (a, b)`: a has the first k elements of t, b the rest.
- `merge(a, b) → t`: concatenation, assuming every element of a precedes every element of b.

`merge` keeps the root with higher priority and recurses into the side that touches the other tree; `split` decides at each node whether the node goes left or right and recurses down one side. Both touch one root-to-leaf path: expected O(log n).

With split and merge, insert/erase/range operations are compositions:

Operation	Implementation
insert v at position p	`split(t, p) → a, b`; `t = merge(merge(a, node(v)), b)`
erase position p	`split(t, p−1) → a, b`; `split(b, 1) → x, c`; `t = merge(a, c)`
operate on [l, r]	split out the middle piece `b`, apply a lazy tag to `b`'s root, merge back

4.3 Implicit keys

In an **implicit treap** we never store keys. The key of a node is its **position in the in-order traversal**, which equals `size(left subtree) + 1` relative to its subtree. Since `split` by count only uses subtree sizes, positions are maintained automatically even when we insert in the middle or cut and paste whole blocks. The implicit treap is therefore a **sequence** data structure ("array with O(log n) insert, delete, cut, paste, reverse").

4.4 Lazy tags

Lazy propagation (week 25) works as usual: each node carries a tag (add, assign, reverse), `push(x)` applies the tag to the children before descending, and `pull(x)` recomputes size and aggregates from the children. The only new tag is **reverse**: reversing a subtree means swapping the left and right child of every node in it. Lazily: toggle a `rev` flag; when pushing, swap the node's children and toggle the flag on both children. Aggregates that are not symmetric (e.g. "best prefix sum") must also swap their prefix/suffix fields on reverse.

Rule: call `push(x)` before reading or changing `x`'s children; call `pull(x)` after changing them. In `split` and `merge` this is at the start and the end of each call, respectively.

4.5 Typical problems

- CSES 2072 — Cut and Paste: move substring [a, b] to the end. Two splits, three merges.
- CSES 2073 — Substring Reversals and CSES 2074 — Reversals and Sums: reverse tag, plus sum aggregate.
- Rope-like problems: "insert a block of text at position p, print character at position q".
- Keyed treaps (ordinary BST) with split/merge also give an easy mergeable ordered set: union of two sets in expected O(m log(n/m)).

5. Choosing between them

Need	First choice	Notes
k-th smallest in range, online, static array	persistent segtree	O(log n) query
count ≤ x in range, offline	sort + Fenwick	simplest, fastest
count ≤ x in range, online, static	merge-sort tree or persistent	MST is easier to adapt
count ≤ x in range, with point updates	BIT/segtree of ordered sets, or sqrt decomposition	$O(\log^2 n)$
old versions / O(1) copy	persistent segtree	
insert/delete/reverse in the middle	implicit treap	

6. Reference implementations

All programs below compile with `g++ -std=c++17 -O2 -Wall` and were stress-tested against brute force on random small inputs.

6.1 Persistent segment tree: k-th smallest in a range (SPOJ MKTHNUM)

Input: `n q`, the array, then `q` lines `l r k` (1-indexed).

```
#include <bits/stdc++.h>
using namespace std;

// Persistent segment tree over compressed values.
// root[i] = tree containing the multiset {a[1..i]}.
struct PST {
    vector<int> L, R, cnt;
    PST(int reserve) { L.reserve(reserve); R.reserve(reserve); cnt.reserve(reserve);
                       L.push_back(0); R.push_back(0); cnt.push_back(0); } // node 0 =
empty tree
    int newNode(int l, int r, int c) { L.push_back(l); R.push_back(r); cnt.push_back(c);
return (int)cnt.size() - 1; }
    // returns new root = old tree with +1 at position pos
    int insert(int prev, int lo, int hi, int pos) {
        if (lo == hi) return newNode(0, 0, cnt[prev] + 1);
        int mid = (lo + hi) / 2;
        if (pos <= mid) { int nl = insert(L[prev], lo, mid, pos); return newNode(nl,
R[prev], cnt[prev] + 1); }
        else            { int nr = insert(R[prev], mid + 1, hi, pos); return
newNode(L[prev], nr, cnt[prev] + 1); }
    }
    // k-th smallest (1-indexed) in (version u) minus (version v)
    int kth(int u, int v, int lo, int hi, int k) {
        while (lo < hi) {
            int mid = (lo + hi) / 2;
            int leftCnt = cnt[L[u]] - cnt[L[v]];
            if (k <= leftCnt) { u = L[u]; v = L[v]; hi = mid; }
            else { k -= leftCnt; u = R[u]; v = R[v]; lo = mid + 1; }
        }
        return lo;
    }
};

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];
    vector<int> vals(a.begin() + 1, a.end());
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
    int m = vals.size();
```

```
    PST t(n * 20 + 5);
    vector<int> root(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        int pos = lower_bound(vals.begin(), vals.end(), a[i]) - vals.begin();
        root[i] = t.insert(root[i - 1], 0, m - 1, pos);
    }
    while (q--) {
        int l, r, k;
        cin >> l >> r >> k;
        cout << vals[t.kth(root[r], root[l - 1], 0, m - 1, k)] << "\n";
    }
}
```

Node 0 is a self-loop "empty tree" (`L[0] = R[0] = 0`, `cnt[0] = 0`), so we never need null checks. Note that `newNode` is called only after the recursive call returns, so the `push_back`s never invalidate a value we still need.

6.2 Persistent point-assign / range-sum with copies (CSES 1737)

```
#include <bits/stdc++.h>
using namespace std;

// CSES 1737 "Range Queries and Copies": persistent point-assign / range-sum.
const int MAXNODES = 200000 * 2 + 200000 * 20 + 10;
int lc[MAXNODES], rc[MAXNODES], cnt = 0;
long long sm[MAXNODES];
int n;

int build(const vector<long long>& a, int lo, int hi) {
    int v = ++cnt;
    if (lo == hi) { sm[v] = a[lo]; return v; }
    int mid = (lo + hi) / 2;
    lc[v] = build(a, lo, mid); rc[v] = build(a, mid + 1, hi);
    sm[v] = sm[lc[v]] + sm[rc[v]];
    return v;
}
int update(int prev, int lo, int hi, int pos, long long x) {
    int v = ++cnt;                          // copy the node, then modify the copy
    lc[v] = lc[prev]; rc[v] = rc[prev];
    if (lo == hi) { sm[v] = x; return v; }
    int mid = (lo + hi) / 2;
    if (pos <= mid) lc[v] = update(lc[prev], lo, mid, pos, x);
    else            rc[v] = update(rc[prev], mid + 1, hi, pos, x);
    sm[v] = sm[lc[v]] + sm[rc[v]];
    return v;
}
long long query(int v, int lo, int hi, int l, int r) {
    if (r < lo || hi < l) return 0;
    if (l <= lo && hi <= r) return sm[v];
    int mid = (lo + hi) / 2;
    return query(lc[v], lo, mid, l, r) + query(rc[v], mid + 1, hi, l, r);
}
```

```
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int q;
    cin >> n >> q;
    vector<long long> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];
    vector<int> roots = {build(a, 1, n)};   // array 1 -> roots[0]
    while (q--) {
        int type, k;
        cin >> type >> k; k--;
        if (type == 1) { int p; long long x; cin >> p >> x; roots[k] = update(roots[k],
1, n, p, x); }
        else if (type == 2) { int l, r; cin >> l >> r; cout << query(roots[k], 1, n, l,
r) << "\n"; }
        else roots.push_back(roots[k]);       // O(1) copy: share the whole tree
    }
}
```

6.3 Merge-sort tree: count of values ≤ x in a range

```
#include <bits/stdc++.h>
using namespace std;

// Merge-sort tree: node stores the sorted values of its segment.
// countLE(l, r, x) = number of i in [l, r] with a[i] <= x, in O(log^2 n).
struct MergeSortTree {
    int n;
    vector<vector<int>> t;
    MergeSortTree(const vector<int>& a) : n(a.size()), t(4 * a.size()) { build(1, 0, n -
1, a); }
    void build(int v, int lo, int hi, const vector<int>& a) {
        if (lo == hi) { t[v] = {a[lo]}; return; }
        int mid = (lo + hi) / 2;
        build(2 * v, lo, mid, a);
        build(2 * v + 1, mid + 1, hi, a);
        merge(t[2 * v].begin(), t[2 * v].end(), t[2 * v + 1].begin(), t[2 * v + 1].end(),
              back_inserter(t[v]));
    }
    int countLE(int v, int lo, int hi, int l, int r, int x) const {
        if (r < lo || hi < l) return 0;
        if (l <= lo && hi <= r)
            return upper_bound(t[v].begin(), t[v].end(), x) - t[v].begin();
        int mid = (lo + hi) / 2;
        return countLE(2 * v, lo, mid, l, r, x) + countLE(2 * v + 1, mid + 1, hi, l, r,
x);
    }
    int countLE(int l, int r, int x) const { return countLE(1, 0, n - 1, l, r, x); }
};

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, q;
```

```
    cin >> n >> q;
    vector<int> a(n);
    for (auto& x : a) cin >> x;
    MergeSortTree mst(a);
    while (q--) {
        int l, r, x;              // 1-indexed, inclusive
        cin >> l >> r >> x;
        cout << mst.countLE(l - 1, r - 1, x) << "\n";
    }
}
```

For KQUERY ("how many > k"), print `(r − l + 1) − countLE(l, r, k)`.

6.4 Implicit treap with reverse and range sum

Operations: `1 l r` reverse, `2 l r` print sum, `3 p v` insert v after the first p elements, `4 p` erase the p-th element.

```
#include <bits/stdc++.h>
using namespace std;

// Implicit treap: a sequence supporting split/merge by position,
// range reverse and range sum, all in expected O(log n).
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

struct Node {
    int l = 0, r = 0, sz = 0;
    unsigned pri = 0;
    long long val = 0, sum = 0;
    bool rev = false;
};
vector<Node> T(1);  // T[0] is the null node (sz = 0, sum = 0)

int newNode(long long v) {
    Node x; x.sz = 1; x.pri = rng(); x.val = x.sum = v;
    T.push_back(x);
    return (int)T.size() - 1;
}
void pull(int x) {
    T[x].sz = 1 + T[T[x].l].sz + T[T[x].r].sz;
    T[x].sum = T[x].val + T[T[x].l].sum + T[T[x].r].sum;
}
void push(int x) {
    if (x && T[x].rev) {
        swap(T[x].l, T[x].r);
        if (T[x].l) T[T[x].l].rev ^= 1;
        if (T[x].r) T[T[x].r].rev ^= 1;
        T[x].rev = false;
    }
}
// split x into a = first k elements, b = the rest
void split(int x, int k, int& a, int& b) {
```

```
    if (!x) { a = b = 0; return; }
    push(x);
    if (T[T[x].l].sz < k) { split(T[x].r, k - T[T[x].l].sz - 1, T[x].r, b); a = x; }
    else                  { split(T[x].l, k, a, T[x].l); b = x; }
    pull(x);
}
int merge(int a, int b) {
    if (!a || !b) return a ? a : b;
    if (T[a].pri > T[b].pri) { push(a); T[a].r = merge(T[a].r, b); pull(a); return a; }
    else                     { push(b); T[b].l = merge(a, T[b].l); pull(b); return b; }
}

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    int root = 0;
    for (int i = 0; i < n; i++) { long long v; cin >> v; root = merge(root, newNode(v));
}
    while (q--) {
        int type; cin >> type;
        if (type == 1 || type == 2) {          // 1 l r: reverse, 2 l r: sum (1-indexed)
            int l, r; cin >> l >> r;
            int a, b, c;
            split(root, l - 1, a, b);
            split(b, r - l + 1, b, c);
            if (type == 1) T[b].rev ^= 1;
            else cout << T[b].sum << "\n";
            root = merge(merge(a, b), c);
        } else if (type == 3) {                 // 3 p v: insert v after the first p
elements
            int p; long long v; cin >> p >> v;
            int a, b;
            split(root, p, a, b);
            root = merge(merge(a, newNode(v)), b);
        } else {                                // 4 p: erase the p-th element
            int p; cin >> p;
            int a, b, c;
            split(root, p - 1, a, b);
            split(b, 1, b, c);
            root = merge(a, c);
        }
    }
}
```

The reverse tag is "pending for the children": the node itself is already correct (a reversed subtree has the same sum), and the swap happens when the node is pushed. Reading in-order (e.g. to print the final sequence) must also `push` on the way down.

7. Common mistakes

- **Persistent tree with implicit `2v, 2v+1` indexing.** Shared children make this impossible; use explicit child arrays.
- **Under-allocating nodes.** A persistent tree needs $\sim n \cdot 2 + q \cdot (\log_2 n + 2)$ nodes. Count before you code; a `vector` with `reserve` avoids both overflow and reallocation surprises.
- **Modifying a shared node in place** (e.g. applying a lazy tag to an old node). In persistent trees with lazy propagation, `push` must *copy* children before modifying them, or use non-propagating "permanent" tags that are added up along the query path.
- **Off-by-one between versions**: the subarray [l, r] is `root[r] − root[l−1]`, so `root[0]` must be the empty tree.
- **Merge-sort tree memory:** 4n vectors plus n log n ints; for $n = 10^6$ this is too much — consider the offline Fenwick approach.
- **Treap: forgetting `push` before reading children** in split, merge, or any walk; or forgetting `pull` after changing them. Symptom: correct on small tests, wrong after a reverse.
- **Treap priorities from `rand()`.** On some platforms `RAND_MAX` = 32767, so many priorities collide and the treap degenerates. Use `mt19937`.
- **Recursion depth.** Treap recursion is O(log n) expected, which is fine; but building a treap of 10^6 elements by repeated `merge` is O(n log n). It is fast enough; an O(n) stack-based build exists if needed.
- **Choosing persistence when the problem is offline.** A sort + Fenwick sweep is shorter and faster.

8. Practice problems

Persistent segment trees

1. CSES 1737 — Range Queries and Copies
2. SPOJ MKTHNUM — K-th Number
3. SPOJ DQUERY — D-query (solve it online with persistence, and offline with a Fenwick tree)
4. Codeforces 707D — Persistent Bookcase (version tree; also solvable offline by DFS over versions)
5. Codeforces 813E — Army Creation (online, k-th previous occurrence)
6. Codeforces 484E — Sign on Fence (binary search over versions)

Merge-sort trees / range × value 7. SPOJ KQUERY — K-query (merge-sort tree and offline Fenwick; compare)

Treaps 8. CSES 2072 — Cut and Paste 9. CSES 2073 — Substring Reversals 10. CSES 2074 — Reversals and Sums 11. Codeforces 863D — Yet Another Array Queries Problem (treap, or reason backwards) 12. Codeforces 702F — T-Shirts (hard: treap with lazy tags and a clever amortized merge)

9. Further reading

- cp-algorithms: Treap, Segment tree — persistent version (section "Persistent Segment Tree").
- USACO Guide: Persistent data structures and Treaps.
- Driscoll, Sarnak, Sleator, Tarjan, "Making data structures persistent" (1989) — the original path-copying / node-copying paper.
- Seidel & Aragon, "Randomized search trees" (1996) — treap analysis.
- Next: Lecture 3 uses a Li Chao tree (a segment tree over x-coordinates storing lines); Lecture 4 combines segment trees with heavy-light decomposition. The World Finals track (worldfinals-04) extends the "split/merge on a sequence" idea to link-cut trees.